



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

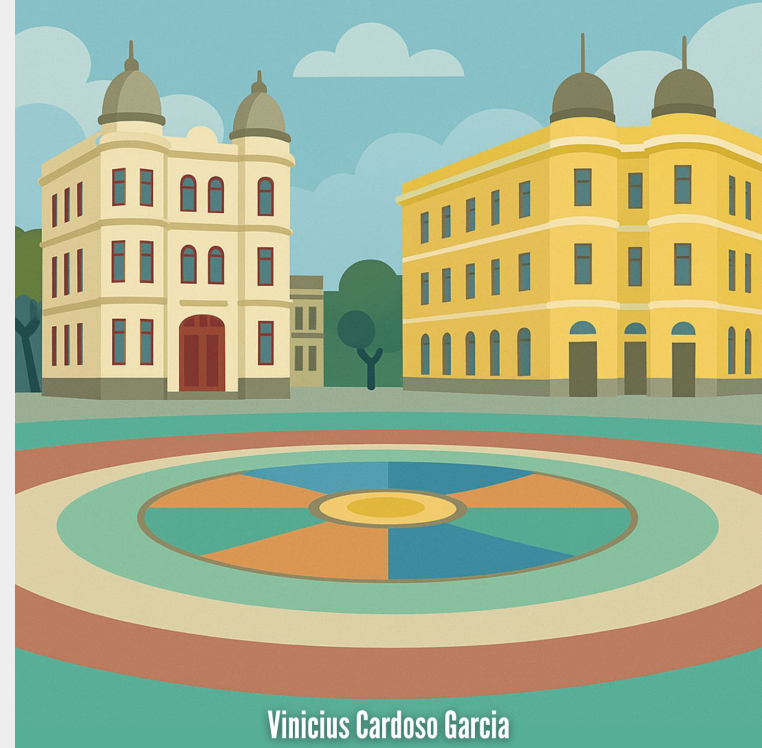
É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE





Testes de Aceitação, BDD e Gherkin

Como conectar user stories e critérios de aceitação a testes formais, transformando requisitos em especificações executáveis com Playwright.

Thion:

- Test ther rocleoings
- Tost orepies
- Test fihe sienocleating
- Test cost baff
- Test che neratms
- Aşpening coft
- Chans
- Thy slen heations



Agenda

01

Testes de Aceitação

O que são, por que importam e como se encaixam no ciclo de desenvolvimento ágil.

03

Formato Gherkin

Estrutura Dado/Quando/Então como especificação executável derivada do backlog.

02

BDD – Behavior-Driven Development

A ponte entre requisitos de negócio, critérios de aceitação e testes automatizados.

04

Cenários e Automação com Playwright

Mapeando cenários de sucesso e falha e implementando testes end-to-end.





O Que São Testes de Aceitação?

Testes de aceitação validam se um sistema satisfaz os **critérios acordados com o cliente ou product owner** antes de uma entrega ser considerada concluída. Eles respondem a uma pergunta simples: "O *software faz o que foi prometido?*"

Perspectiva de Negócio

Baseados em requisitos funcionais e regras de negócio definidas pelo PO.

Critério de Pronto

Definem o "Definition of Done" técnico e funcional de cada user story.

Validação Final

Última barreira antes do deploy — garantia de que o comportamento esperado foi entregue.



Por Que Testes de Aceitação Importam?

Sem Testes de Aceitação

- Requisitos ambíguos causam retrabalho
- Bugs descobertos apenas em produção
- Critério de "pronto" subjetivo
- Comunicação falha entre dev, QA e PO
- Releases inseguros e imprevisíveis

Com Testes de Aceitação

- Requisitos documentados de forma objetiva
- Bugs capturados antes da produção
- Critério de "pronto" claro e mensurável
- Alinhamento contínuo entre todas as partes
- Confiança para fazer releases frequentes



Pirâmide de Testes: Onde Vivem os Testes de Aceitação?

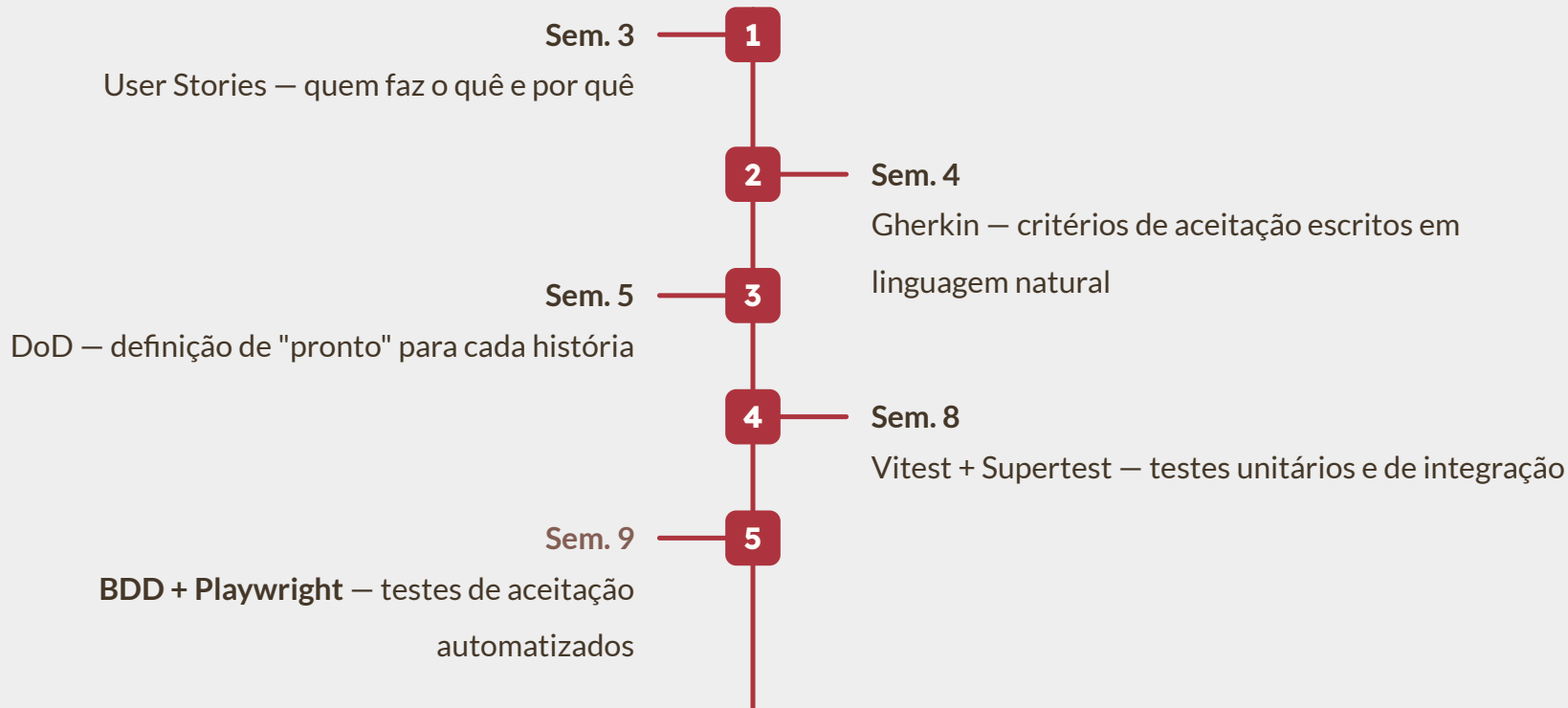


Testes de aceitação ficam no topo da pirâmide: são em menor quantidade, porém cobrem os fluxos de maior valor para o negócio. Eles validam o comportamento completo do sistema do ponto de vista do usuário, complementando os testes unitários e de integração que garantem a saúde interna do código.



De Requisito a Teste Automatizado

Cada semana deste semestre construiu sobre a anterior. Hoje conectamos todas as peças do ciclo.



BDD: A Ponte Entre Requisitos e Código

Behavior-Driven Development (BDD) é uma abordagem de desenvolvimento que estende o TDD ao incorporar a linguagem de negócio. Em vez de descrever testes técnicos, o BDD descreve *comportamentos esperados do sistema* em linguagem natural estruturada.

1

Requisitos

User stories e critérios definidos pelo PO

2

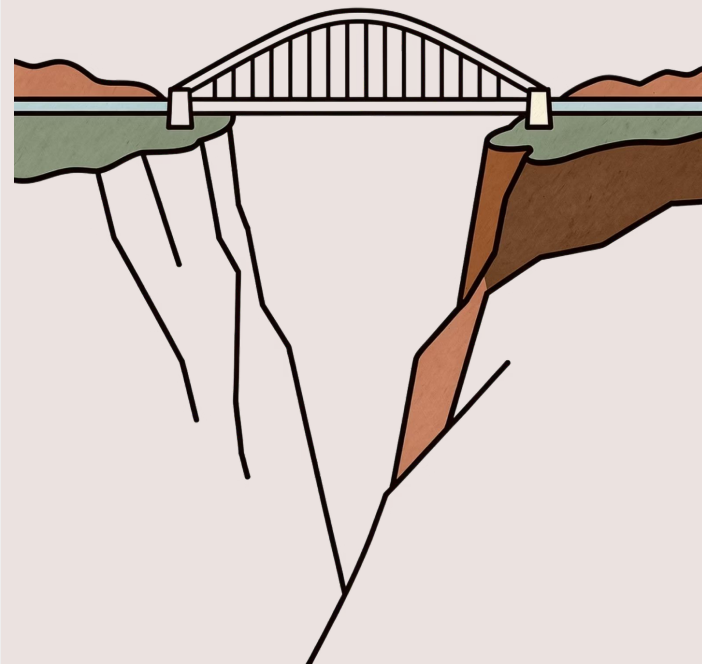
Cenários BDD

Especificações em linguagem Gherkin

3

Testes Automatizados

Código executável que valida cada cenário



Behavior-Driven Development

BDD não é só uma técnica de teste — é uma linguagem compartilhada entre times técnicos e não-técnicos.

Comportamento primeiro

O comportamento esperado é especificado **antes** do código ser escrito, guiando a implementação.

Linguagem compartilhada

Devs e stakeholders leem e entendem os mesmos cenários, sem tradução necessária.

Nível de aceitação

BDD **não substitui** testes unitários, ele atua no nível mais alto, validando o sistema como um todo.

BDD: "O sistema se **COMPORTA** como esperado?" · TDD: "A função **FUNCIONA** corretamente?"



Os Três Pilares do BDD



Conversas

BDD começa com diálogo entre PO, dev e QA, os "três amigos", para alinhar expectativas antes de qualquer linha de código.



Especificações

As conversas se tornam cenários formais em Gherkin, documentação viva que todos os papéis conseguem ler e validar.



Automação

Os cenários são automatizados e executados a cada build, garantindo que o comportamento especificado continua funcionando.



Os "Três Amigos" do BDD

Uma prática central do BDD é a **reunião dos três amigos** — uma conversa breve e focada antes do início do desenvolvimento de cada história.



Product Owner

Traz o "o quê", o valor de negócio, as regras e os critérios de aceitação da story.



Desenvolvedor

Questiona viabilidade técnica e identifica casos extremos que o PO pode não ter considerado.



QA Engineer

Foca nos cenários de falha, bordas e comportamentos inesperados que devem ser cobertos.



Gherkin: A Linguagem de Especificação Executável

Gherkin é a linguagem estruturada usada para escrever cenários BDD.

Seu design permite que **pessoas de negócio leiam e validem** os cenários enquanto ferramentas de automação os executam

Legível por Humanos

Usa palavras do domínio de negócio, sem jargão técnico. Qualquer stakeholder consegue revisar um cenário Gherkin.

Executável por Máquinas

Ferramentas como Cucumber, Behave e o próprio Playwright interpretam a estrutura Gherkin e a mapeiam para código.

Documentação Viva

O cenário nunca fica desatualizado — se o comportamento muda, o teste falha, sinalizando que a documentação precisa de revisão.



Estrutura de um Arquivo Gherkin

Palavras-Chave Principais

1 Feature
Descreve a funcionalidade de alto nível sendo especificada.
Equivale a uma épica ou user story maior.

2 Scenario
Um caso de uso concreto com contexto, ação e resultado esperado. Cada cenário é independente.

3 Background
Passos comuns a todos os cenários de uma Feature, executados antes de cada Scenario.

Exemplo de Arquivo .feature

```
Feature: Login de usuário  
  Como usuário registrado  
  Quero fazer login no sistema  
  Para acessar minha conta
```

```
Background:  
  Given que a página de login está aberta
```

```
Scenario: Login com credenciais válidas  
  When informo email "user@email.com"  
  And informo senha "Senha@123"  
  And clico em "Entrar"  
  Then devo ver o dashboard
```

```
Scenario: Login com senha inválida  
  When informo email "user@email.com"  
  And informo senha "errada"  
  And clico em "Entrar"  
  Then devo ver "Credenciais inválidas"
```



Dado • Quando • Então

A tríade fundamental do Gherkin que estrutura qualquer cenário de teste de aceitação.



Dado / Quando / Então — Detalhado

Dado (Given)

Contexto inicial. Descreve o estado do sistema antes da ação ocorrer. Responde: "Em que situação o usuário está?"

Ex.: Dado que o usuário está logado como administrador

Quando (When)

A ação ou evento. Descreve o que o usuário ou sistema faz. Responde: "O que acontece?"

Ex.: Quando ele clica em "Excluir produto"

Então (Then)

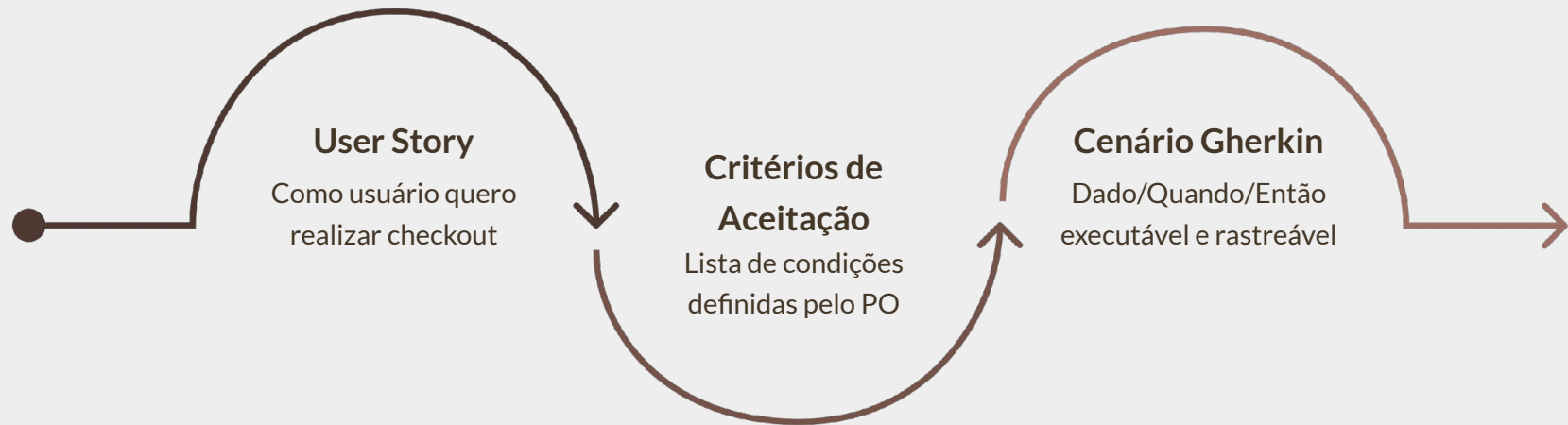
O resultado esperado. Descreve o comportamento observável após a ação. Responde: "O que deve ser verdade?"

Ex.: Então o produto não aparece mais na listagem

- ❗ A palavra-chave **E (And)** pode ser usada após qualquer uma das três para encadear múltiplos passos do mesmo tipo sem repetir a palavra-chave.



De User Story ao Cenário Gherkin



Cada critério de aceitação de uma user story deve se tornar pelo menos um cenário Gherkin. Isso cria rastreabilidade direta entre o backlog e os testes — qualquer história sem cenário associado ainda não tem critério de "pronto" verificável.



Exemplo Prático: Carrinho de Compras

User Story de Origem

Como cliente, quero adicionar produtos ao carrinho para poder comprá-los depois.

Cenários Gerados

Feature: Gerenciamento do carrinho de compras

Scenario: Adicionar produto disponível ao carrinho

Given que estou na página do produto "Tênis Running X"

And o produto tem estoque disponível

When clico no botão "Adicionar ao carrinho"

Then o contador do carrinho deve aumentar em 1

And devo ver a notificação "Produto adicionado"

Scenario: Tentar adicionar produto sem estoque

Given que estou na página do produto "Câmera Y"

And o produto está sem estoque

Then o botão "Adicionar ao carrinho" deve estar desabilitado



Cenários de Sucesso vs. Cenários de Falha

Um conjunto de testes robusto precisa cobrir **ambos os lados** de cada comportamento. Focar apenas no caminho feliz é a causa mais comum de bugs que chegam à produção.

Cenários de Sucesso (Happy Path)

- Fluxo principal com dados válidos
- Usuário com permissões corretas
- Sistema em estado esperado
- Todas as dependências disponíveis

Cenários de Falha (Sad Path)

- Dados inválidos ou malformados
- Usuário sem permissão adequada
- Recurso não encontrado (404)
- Timeout, serviço indisponível
- Campos obrigatórios em branco



O Caminho Feliz

O cenário de sucesso descreve o fluxo ideal, quando tudo funciona como o usuário espera. É o ponto de partida obrigatório.

Cenário: Filtro com resultados

Dado que estou na página de listagem de produtos

E existem 20 produtos com preços variados no catálogo

Quando seleciono a faixa de preço "R\$50 a R\$100"

E cliço no botão "Filtrar"

Então apenas produtos entre R\$50 e R\$100 são exibidos na tela

E a contagem de resultados é atualizada corretamente

Cada passo é preciso, verificável e livre de ambiguidade. Se o time não concorda sobre o resultado esperado, o cenário revela isso antes de qualquer linha de código.



Quando Dá Errado ✖

O cenário de falha é tão importante quanto o de sucesso. Ele especifica o que o sistema deve fazer — não apenas o que deixa de fazer.

✖ Cenário: Filtro sem resultados

Dado que todos os produtos no catálogo custam mais de R\$500

Quando seleciono a faixa de preço "R\$10 a R\$50"

E cliço no botão "Filtrar"

Então nenhum produto é exibido na listagem

E a mensagem "*Nenhum produto encontrado*" aparece em destaque na tela

"Se vocês só especificaram o caminho feliz, especificaram **metade** do comportamento do sistema."



Para Cada Sucesso, Pelo Menos Uma Falha

Quatro perguntas que, quando feitas antes de codificar, revelam cenários inteiros que ninguém havia pensado.



E se o campo estiver vazio?

Campos obrigatórios sem preenchimento devem ter feedback claro e imediato ao usuário.



E se não houver permissão?

Ações restritas precisam retornar erro adequado — não silêncio ou crash.



E se o dado não existir?

Registros inexistentes exigem tratamento explícito — a tela de erro também é parte do produto.

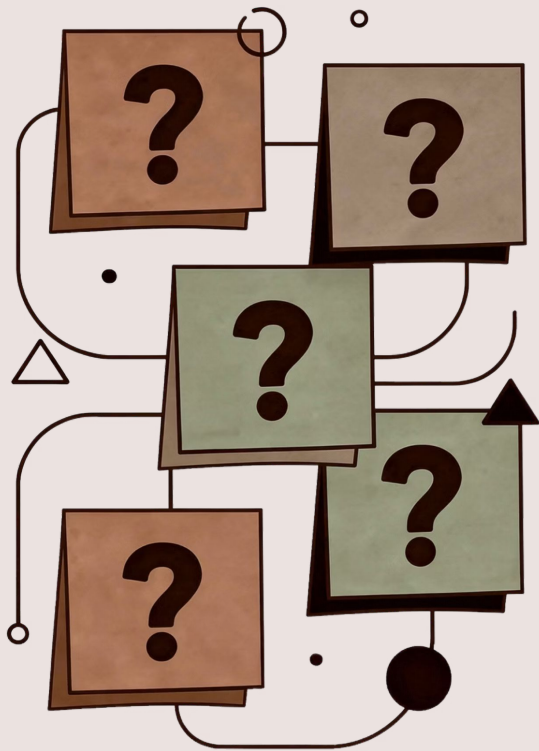


E se a rede cair?

Falhas de conectividade no meio de uma ação são cenários reais que precisam de resposta.



"O stakeholder na quinta vai perguntar: 'e se...?' Se vocês não testaram, não sabem a resposta."



Boas Práticas na Escrita de Cenários



Um comportamento por cenário

Cenários que testam muitas coisas ao mesmo tempo são difíceis de diagnosticar quando falham. Mantenha cada cenário focado em um único comportamento observável.



Use linguagem do domínio, não da UI

Prefira "o usuário realiza o login" a "o usuário clica no botão id='btn-submit'". Cenários ligados a detalhes de UI quebram a cada refatoração visual.



Evite dependência entre cenários

Cada cenário deve poder ser executado de forma isolada. Dependências entre cenários criam falhas em cascata difíceis de rastrear.



O Teste Documenta Decisões

O login parece simples — mas três cenários revelam uma decisão arquitetural de segurança que precisa estar documentada.

✓ Sucesso — credenciais corretas

Email e senha válidos → usuário redirecionado para a página inicial com sessão ativa.

✗ Falha — senha incorreta

Credenciais com senha errada → exibe *"Email ou senha incorretos"*.

✗ Falha — conta inexistente

Email não cadastrado → exibe a **mesma** mensagem: *"Email ou senha incorretos"*.

Por que a mensagem é idêntica?

Segurança: revelar se um email está cadastrado é uma vulnerabilidade. O teste documenta essa decisão consciente da equipe.

O valor da documentação em cenários

Sem o cenário explícito, o próximo dev poderia "corrigir" essa mensagem como se fosse um bug. O BDD protege decisões intencionais.



Scenario Outline: Parametrizando Cenários

Quando o mesmo comportamento precisa ser testado com múltiplos conjuntos de dados, o **Scenario Outline** (ou Scenario Template) elimina duplicação usando uma tabela de exemplos.

Scenario Outline: Validação de campos do formulário de login

Given que estou na página de login

When informo o email "<email>" e senha "<senha>"

And clico em "Entrar"

Then devo ver a mensagem "<mensagem>"

Examples:

email	senha	mensagem
" "	"abc123"	"E-mail é obrigatório"
"invalido"	"abc123"	"E-mail inválido"
"a@b.com"	" "	"Senha é obrigatória"
"a@b.com"	"abc123"	"Credenciais inválidas"

- ❏ Cada linha da tabela **Examples** gera um cenário independente na execução dos testes.



Playwright: Automação E2E Moderna

Playwright é uma ferramenta open-source criada pela Microsoft para automação de testes end-to-end. Ela controla browsers reais (Chromium, Firefox, WebKit) programaticamente, simulando interações reais de usuário com a aplicação.

Multi-browser

Executa os mesmos testes em Chromium, Firefox e Safari com uma única API.

Auto-wait

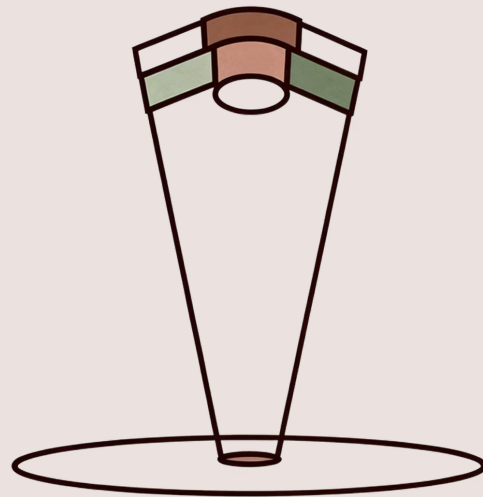
Aguarda elementos estarem prontos automaticamente, eliminando sleeps e timeouts frágeis.

TypeScript Nativo

Suporte completo a TypeScript com tipagem forte e IntelliSense no editor.

Tracing e Debug

Grava screenshots, vídeos e traces de execução para facilitar diagnóstico de falhas.



Playwright: Automação que Simula o Usuário Real

O Playwright é a ferramenta que transforma os cenários BDD em testes executáveis, ele não testa funções, ele testa a experiência completa no navegador.



Open-source da Microsoft

Mantido ativamente, amplamente adotado na indústria e com excelente documentação oficial.



Simula ações humanas reais

Clicar, digitar, navegar entre páginas, aguardar elementos carregarem — tudo como um usuário faria.



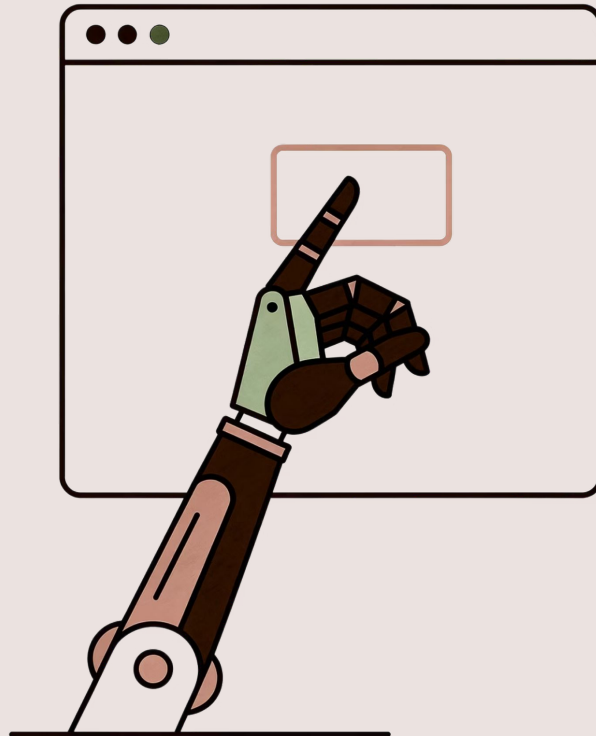
Chrome, Firefox e Safari

Testa nos três principais navegadores com uma única base de código, garantindo compatibilidade.



Rápido e confiável

Execução em segundos, espera automática por elementos e relatórios com screenshots em caso de falha.



Cada Ferramenta no Seu Lugar

Vitest e Playwright não competem, eles se complementam. Entender onde cada um atua é essencial para construir uma suíte de testes sólida.

	Vitest	Playwright
Nível	Unitário / Integração	E2E / Aceitação
O que testa	Funções, módulos, rotas de API	Aplicação completa no navegador
Velocidade	Milissegundos	Segundos
Ambiente	Node.js (sem navegador)	Navegador real (Chrome/Firefox/Safari)
Pergunta central	"A lógica funciona?"	"A <i>experiência</i> funciona?"

 Use Vitest para cobrir a lógica de negócio com velocidade. Use Playwright para validar os fluxos que o usuário realmente percorre.



Por Que Playwright para Testes de Aceitação?

3

Engines de browser

Chromium, Firefox e WebKit cobertos com a mesma suíte de testes.

0

sleeps manuais

O mecanismo de auto-wait torna os testes confiáveis sem esperas hardcoded.

100%

open-source

Mantido pela Microsoft com comunidade ativa e atualizações frequentes.

Além dessas vantagens técnicas, o Playwright possui integração nativa com o padrão **Page Object Model (POM)**, facilitando a organização dos testes e a reutilização de código — um requisito para suítes de aceitação sustentáveis em projetos reais.



Estrutura de um Teste Playwright

```
import { test, expect } from '@playwright/test';

test.describe('Feature: Login de usuário', () => {

  test.beforeEach(async ({ page }) => {
    // Given: a página de login está aberta
    await page.goto('/login');
  });

  test('Cenário: Login com credenciais válidas', async ({ page }) => {
    // When
    await page.fill('[data-testid="email"]', 'user@email.com');
    await page.fill('[data-testid="password"]', 'Senha@123');
    await page.click('[data-testid="btn-login"]');

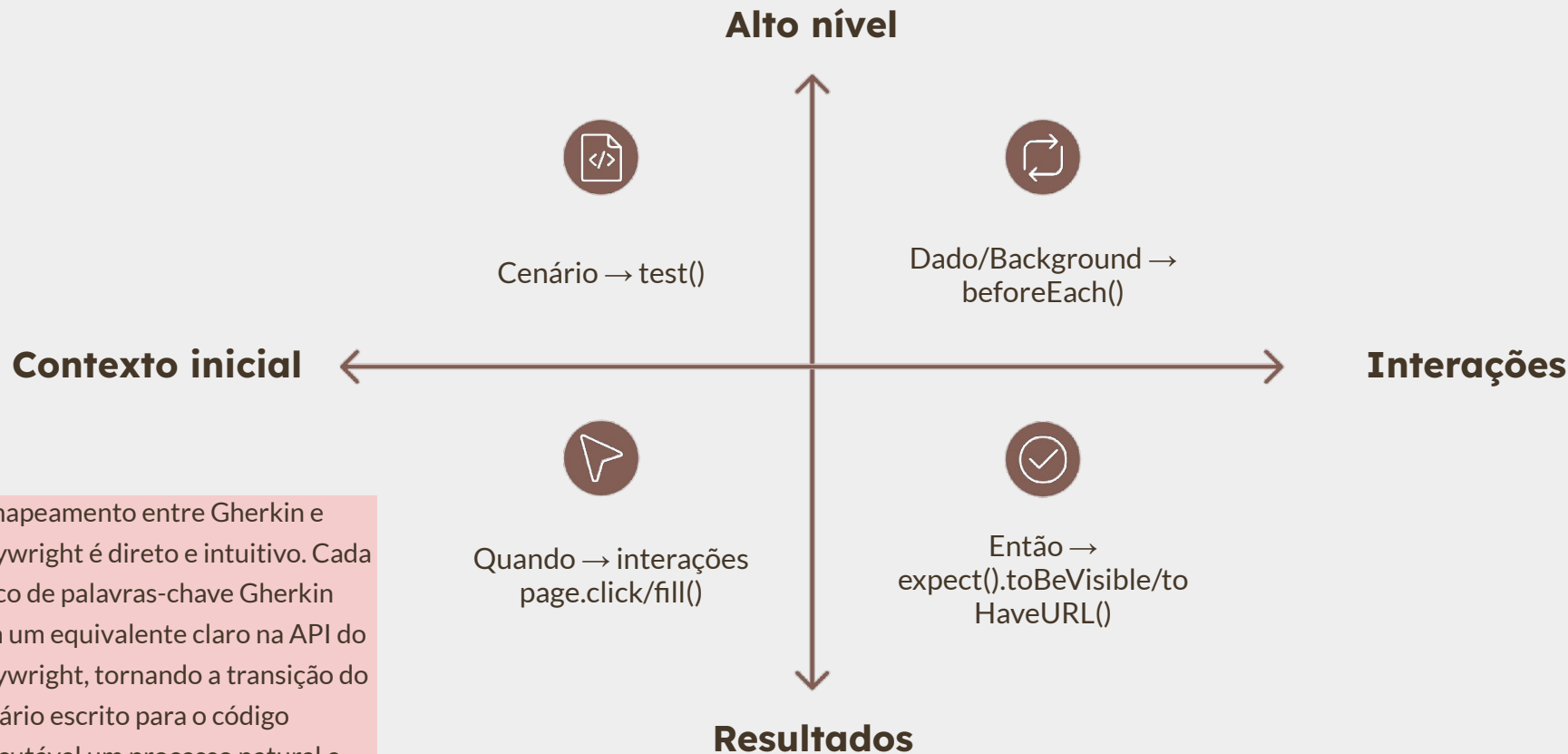
    // Then
    await expect(page).toHaveURL('/dashboard');
    await expect(page.getByText('Bem-vindo!')).toBeVisible();
  });

  test('Cenário: Login com senha inválida', async ({ page }) => {
    await page.fill('[data-testid="email"]', 'user@email.com');
    await page.fill('[data-testid="password"]', 'errada');
    await page.click('[data-testid="btn-login"]');

    await expect(page.getByText('Credenciais inválidas')).toBeVisible();
  });
});
```



Mapeando Gherkin para Playwright



O mapeamento entre Gherkin e Playwright é direto e intuitivo. Cada bloco de palavras-chave Gherkin tem um equivalente claro na API do Playwright, tornando a transição do cenário escrito para o código executável um processo natural e rastreável.

Locators: Encontrando Elementos de Forma Robusta

A qualidade dos seletores é decisiva para testes de aceitação duradouros. O Playwright oferece uma hierarquia de locators — prefira os mais semânticos e resistentes a mudanças de layout.

1

CSS / XPath

Frágeis, quebram com refatorações visuais

2

data-testid

Atributo dedicado a testes, estável e explícito

3

getByRole / getByLabel

Semânticos, refletem a estrutura de acessibilidade da aplicação

4

getByText / getByPlaceholder

Mais próximos da perspectiva do usuário real, ideais para aceitação



Page Object Model com Playwright

Por que usar POM?

O **Page Object Model** encapsula os detalhes de interação de cada página em uma classe dedicada. Os testes de aceitação ficam legíveis, próximos da linguagem do cenário Gherkin, sem expor detalhes técnicos de seletores.

- Reutilização de código entre testes
- Centralização de seletores
- Facilidade de manutenção
- Testes com leitura mais fluida

```
// pages/LoginPage.ts
export class LoginPage {
  constructor(private page: Page) {}

  async goto() {
    await this.page.goto('/login');
  }

  async login(email: string, senha: string) {
    await this.page.fill('[data-testid="email"]', email);
    await this.page.fill('[data-testid="password"]', senha);
    await this.page.click('[data-testid="btn-login"]');
  }

  async getMensagemErro() {
    return this.page.locator('[data-testid="error-msg"]');
  }
}

// test usando o POM
test('login inválido', async ({ page }) => {
  const login = new LoginPage(page);
  await login.goto();
  await login.login('a@b.com', 'errada');
  await expect(await login.getMensagemErro())
    .toBeVisible();
});
```



Cenários de Falha na Prática

Testar apenas o caminho feliz cobre menos de 40% dos comportamentos relevantes de uma aplicação. Veja como mapear falhas comuns em cenários Gherkin e testes Playwright:

Validação de Formulário

*Dado campo obrigatório vazio, Quando submeter o formulário,
Então exibir mensagem de erro inline sem redirecionar.*

Acesso Não Autorizado

*Dado usuário sem permissão de admin, Quando acessar /admin,
Então redirecionar para /403 com mensagem adequada.*

Timeout de API

*Dado serviço de pagamento indisponível, Quando finalizar pedido,
Então exibir mensagem de erro e não duplicar a cobrança.*

Recurso Inexistente

*Dado URL de produto inválido, Quando acessar a página, Então
exibir página 404 personalizada.*



Interceptando Requisições no Playwright

Para testar cenários de falha envolvendo APIs externas sem depender de serviços reais, o Playwright permite **interceptar e mockar requisições HTTP** diretamente no teste.

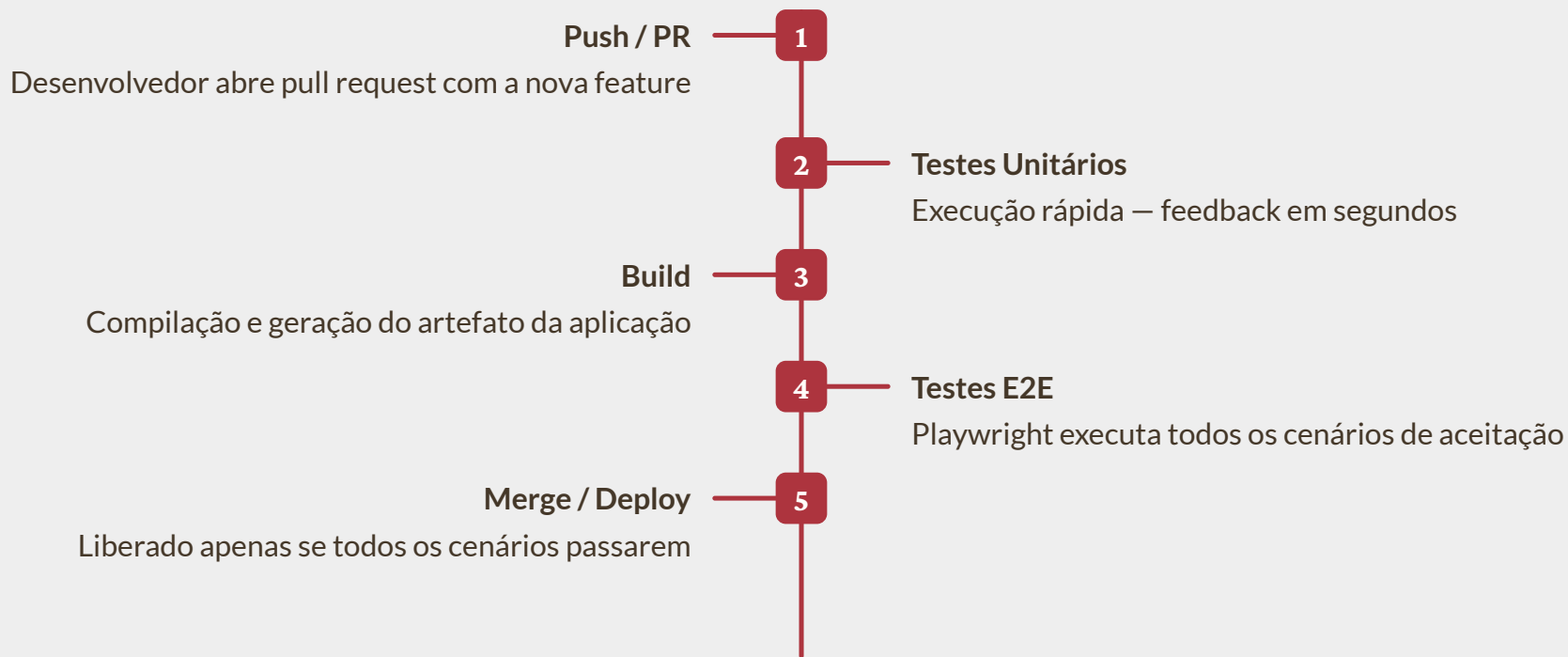
```
test('Cenário: erro de API no checkout', async ({ page }) => {  
  // Intercepta a chamada de pagamento e simula falha  
  await page.route('**/api/pagamento', route =>  
    route.fulfill({  
      status: 503,  
      body: JSON.stringify({ erro: 'Serviço indisponível' })  
    })  
  );  
  
  await page.goto('/checkout');  
  await page.click('[data-testid="btn-finalizar"]');  
  
  // Then: mensagem de erro deve aparecer  
  await expect(  
    page.getByText('Serviço de pagamento indisponível')  
  ).toBeVisible();  
  
  // E o pedido não deve ser criado  
  await expect(page).not.toHaveURL('/pedido/sucesso');  
});
```

✔ Usar `page.route()` torna os testes de cenários de falha **determinísticos** — sem depender de ambientes externos instáveis.



Integrando Testes de Aceitação ao Pipeline CI/CD

Testes de aceitação ganham seu valor máximo quando executados automaticamente a cada pull request. A integração com CI/CD transforma o pipeline em uma **barreira de qualidade contínua**.



Configuração Básica do Playwright

Instalação e Setup

```
# Instalar o Playwright
npm init playwright@latest

# Estrutura gerada
├─ playwright.config.ts
├─ tests/
│   └─ exemplo.spec.ts
└─ package.json
```

Executando os

```
# Modo headless (CI)
npx playwright test

# Com interface visual
npx playwright test --ui

# Arquivo específico
npx playwright test login.spec.ts

# Gerar relatório HTML
npx playwright show-report
```

playwright.config.ts Essencial

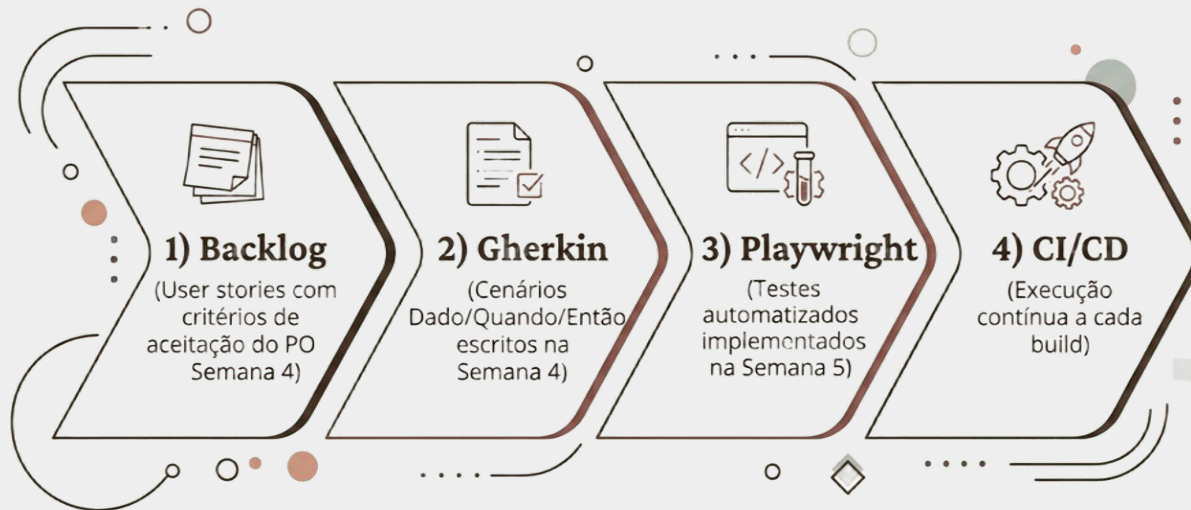
```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  retries: process.env.CI ? 2 : 0,
  use: {
    baseURL: 'http://localhost:3000',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    trace: 'on-first-retry',
  },
  projects: [
    { name: 'chromium' },
    { name: 'firefox' },
    { name: 'webkit' },
  ],
});
```



Conexão: Da Semana 4 ao Teste Automatizado

Os critérios de aceitação em Gherkin escritos na Semana 4 são o ponto de partida direto para os testes automatizados desta semana. O fluxo é completo e rastreável:



✓ Cada critério de aceitação Gherkin da Semana 4 deve resultar em **pelo menos um teste Playwright** — garantindo cobertura total das histórias do projeto.

Checklist: Story Pronta para Testes de Aceitação

Use este checklist antes de iniciar a automação de uma user story do projeto:

Especificação

- ☐ User story tem critérios de aceitação definidos
- ☐ Cada critério tem pelo menos um cenário Gherkin
- ☐ Cenários cobrem happy path e sad paths relevantes
- ☐ Linguagem usa vocabulário do domínio (não da UI)
- ☐ Scenario Outlines usados onde há múltiplos dados

Automação

- ☐ Page Objects criados para as páginas envolvidas
- ☐ Locators usam data-testid ou roles semânticos
- ☐ Cenários de falha mockam dependências externas
- ☐ Testes executam em modo headless sem erros
- ☐ Testes adicionados ao pipeline CI/CD do projeto



Armadilhas Comuns a Evitar



Testes Frágeis (Flaky Tests)

Evite seletores baseados em posição ou classes dinâmicas. Use `data-testid` e o mecanismo de auto-wait do Playwright em vez de `sleeps`.



Dependência entre Testes

Cada teste deve ser independente. Nunca use estado deixado por um teste anterior — use `beforeEach` para garantir um estado inicial limpo.



Cenários Gherkin Técnicos Demais

Gherkin que cita IDs de banco, endpoints ou seletores CSS perde o valor de documentação. Mantenha a linguagem no nível do comportamento do usuário.



Cobrir Só o Happy Path

A maioria dos bugs acontece nos caminhos alternativos. Planeje explicitamente cenários de falha para cada regra de negócio crítica.



Atividade Prática da Semana

Aplicando o conteúdo do módulo diretamente no projeto do time:

01

Revisar os Cenários da Semana 4

Pegue os cenários Gherkin escritos anteriormente e valide se cobrem happy path e pelo menos um sad path para cada critério de aceitação.

02

Configurar o Playwright no Projeto

Execute `npm init playwright@latest` no repositório do projeto e configure o `playwright.config.ts` com a URL base da aplicação.

03

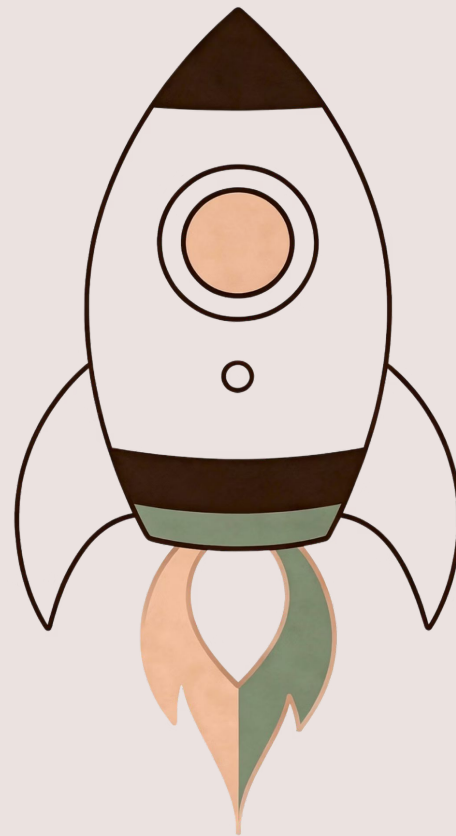
Implementar Page Objects

Crie ao menos uma classe Page Object para as páginas envolvidas nos cenários, encapsulando locators e ações.

04

Automatizar os Cenários

Implemente os testes Playwright mapeando cada bloco Gherkin para os métodos correspondentes. Execute e valide que todos passam.



Recursos e Referências



Documentação Oficial do Playwright

playwright.dev — referência completa de API, guias de início rápido e exemplos de integração com CI/CD.



Cucumber / Gherkin Docs

cucumber.io/docs/gherkin — referência da sintaxe Gherkin, boas práticas de escrita e exemplos por linguagem.



BDD in Action — John Ferguson Smart

Livro de referência sobre BDD com exemplos práticos de como conectar requisitos de negócio a testes automatizados em equipes ágeis.





Resumo: O Que Aprendemos Hoje

Testes de Aceitação

Validam se o software entrega o comportamento prometido, servindo como critério objetivo de "pronto" para cada user story.

BDD

Abordagem que conecta negócio e técnico através de conversas e especificações em linguagem compartilhada entre PO, dev e QA.

Gherkin

Formato Dado/Quando/Então que torna especificações legíveis por humanos e executáveis por ferramentas de automação.

Playwright

Ferramenta moderna de automação E2E que mapeia diretamente os cenários Gherkin para testes robustos e confiáveis.

✅ A rastreabilidade **Backlog** → **Gherkin** → **Playwright** → **CI/CD** é o núcleo de uma estratégia de qualidade contínua em times ágeis.



Vamos debater?



Vamos debater?



Por que testar?



introdução sobre o tema



questões



referências



análises

 **ativar modo anônimo**

o modo anônimo permite responder as questões temporariamente de forma anônima. ao desabilitar, as pessoas que responderam passam a ser identificadas.

introdução sobre o tema

A busca incessante pela excelência em engenharia de software encontra, nos testes de software, uma de suas principais aliadas. Testar não é apenas uma etapa do ciclo de desenvolvimento, mas uma estratégia fundamental para assegurar a qualidade, a confiabilidade e a manutenção dos sistemas entregues. O processo de teste permite identificar defeitos, garantir que os requisitos sejam atendidos e reduzir custos ao capturar problemas antes que eles impactem o usuário final ou a produção. Nesse contexto, entender os diferentes tipos de testes, suas relações e implicações econômicas, é essencial para qualquer equipe de desenvolvimento que almeje entregar produtos robustos e bem-sucedidos.

questão 1

Quais são os principais benefícios e desafios encontrados ao aplicar cada tipo de teste (unitário, integração, sistema e aceitação) no ciclo de desenvolvimento de software?

3 respostas · 3 comentários

questão 2

A busca incessante pela excelência em engenharia de software encontra, nos testes de software, uma de suas principais aliadas. Testar não é apenas uma etapa do ciclo de desenvolvimento, mas uma estratégia fundamental para assegurar a qualidade, a confiabilidade e a manutenção dos sistemas entregues. O processo de teste permite identificar defeitos, garantir que os requisitos sejam atendidos e reduzir custos ao capturar problemas antes que eles impactem o usuário final ou a produção. Nesse contexto, entender os diferentes tipos de testes, suas relações e implicações econômicas, é essencial para qualquer equipe de desenvolvimento que almeje entregar produtos robustos e bem-sucedidos.

Os testes podem ser classificados segundo sua granularidade e objetivo. Testes unitários verificam o menor componente possível — normalmente funções ou métodos — de forma isolada, assegurando que cada parte do sistema se comporta conforme esperado. Em seguida, os testes de integração visam identificar falhas na interação entre módulos distintos, mitigando problemas que surjam da composição de unidades que, individualmente, podem ter passado nos testes. Os testes de sistema compreendem a avaliação do aplicativo em sua totalidade, validando



Testes de aceitação



introdução sobre o tema



questões



referências



análises

 **ativar modo anônimo**

o modo anônimo permite responder as questões temporariamente de forma anônima. ao desabilitar, as pessoas que responderam passam a ser identificadas.

introdução sobre o tema

No contexto do desenvolvimento de software contemporâneo, os testes de aceitação se destacam como instrumentos indispensáveis para a garantia da qualidade e do alinhamento dos produtos com as expectativas dos usuários finais. Esses testes representam uma prática que vai além da simples validação técnica de funcionalidades; sua função primordial é verificar se um sistema efetivamente atende aos requisitos de negócio estabelecidos no início do projeto, servindo como ponte entre as áreas técnicas e os stakeholders não técnicos. A centralidade dos testes de aceitação se reflete na sua capacidade de reduzir riscos relacionados a entregas equivocadas e aumentar a confiança nas soluções desenvolvidas, influenciando diretamente a satisfação do cliente e a evolução sustentável dos projetos.

questão 1

De que maneira os testes de aceitação, escritos em Gherkin, contribuem para o alinhamento entre as expectativas do cliente e a entrega de software?

3 respostas · 3 comentários

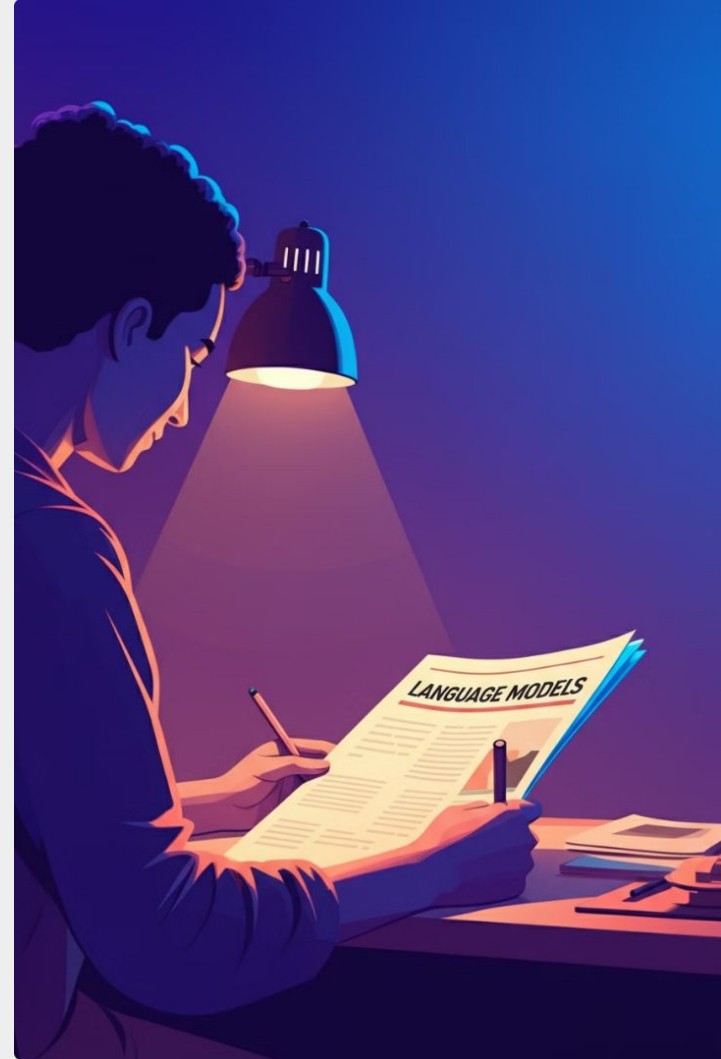
questão 2

No contexto do desenvolvimento de software contemporâneo, os testes de aceitação se destacam como instrumentos indispensáveis para a garantia da qualidade e do alinhamento dos produtos com as expectativas dos usuários finais. Esses testes representam uma prática que vai além da simples validação técnica de funcionalidades; sua função primordial é verificar se um sistema efetivamente atende aos requisitos de negócio estabelecidos no início do projeto, servindo como ponte entre as áreas técnicas e os stakeholders não técnicos. A centralidade dos testes de aceitação se reflete na sua capacidade de reduzir riscos relacionados a entregas equivocadas e aumentar a confiança nas soluções desenvolvidas, influenciando diretamente a satisfação do cliente e a evolução sustentável dos projetos.

Nesse cenário, o Behavior-Driven Development (BDD) emerge como uma abordagem fundamental para conectar requisitos, testes e desenvolvimento, promovendo uma comunicação transparente e colaborativa entre todas as partes envolvidas no processo de desenvolvimento de software. O BDD incentiva a escrita de requisitos de negócio em uma linguagem comum, compreendida tanto por desenvolvedores quanto por analistas e clientes, possibilitando que todos participem ativamente da

Próximos passos

- Eng. Software em Dimensões
 - Cap. 9 (seções 9.1–9.4): Testes: tipos, pirâmide, cobertura, aceitação
- Eng. Software Moderna
 - Cap. 8 — Testes: Unidade, integração, sistema, cobertura, TDD



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
[viniciusgarcia.me](https://www.viniciusgarcia.me)

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Testes de Aceitação, BDD e
Gherkin